Настрой автоматический деплой документации

Настроим автоматический деплой документации через **MkDocs + GitHub Pages**. При каждом `push` в `main` GitHub Actions соберёт сайт из Markdown и опубликует его по адресу:

```
https://YOUR_USERNAME.github.io/responsibility-response-model/
```

GitHub Pages поддерживает деплой через custom workflow: сборка статических файлов, загрузка артефакта и вызов `actions/deploy-pages`.[1] [2] [3]

Обновлённая структура

```
responsibility-response-model/
├── .github/
│   └── workflows/
│       ├── ci.yml
│       └── deploy-docs.yml
├── docs/
│   ├── index.md
│   ├── model.md
│   ├── examples.md
│   └── installation.md
├── rrm/
│   ├── __init__.py
│   ├── model.py
│   ├── scenarios.py
│   ├── visualization.py
│   └── cli.py
├── tests/
│   ├── __init__.py
│   ├── test_model.py
│   └── test_scenarios.py
├── .gitignore
├── mkdocs.yml
├── requirements.txt
├── setup.py
├── README.md
└── LICENSE
```

Файл: .github/workflows/deploy-docs.yml

Создай файл:

```
.github/workflows/deploy-docs.yml
```

И вставь:

```
name: Deploy documentation

on:
  push:
    branches:
      - main
  workflow_dispatch:

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    name: Build documentation
    runs-on: ubuntu-latest

    steps:
      - name: Checkout repository
        uses: actions/checkout@v6

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
          cache: pip

      - name: Install documentation dependencies
        run: |
          python -m pip install --upgrade pip
          pip install mkdocs mkdocs-material mkdocstrings[python]

      - name: Build documentation
        run: |
          mkdocs build --strict

      - name: Upload Pages artifact
        uses: actions/upload-pages-artifact@v4
        with:
          path: ./site
```

```
  deploy:
    name: Deploy documentation
    needs: build

    runs-on: ubuntu-latest

    environment:
      name: github-pages
      url: ${{ steps.deployment.outputs.page_url }}

    steps:
      - name: Deploy to GitHub Pages
        id: deployment
        uses: actions/deploy-pages@v4
```

Workflow состоит из двух jobs: `build` собирает документацию, а `deploy` публикует её; `deploy` зависит от `build` через `needs: build`. Для `deploy-pages` требуются права `pages: write` и `id-token: write`.[2] [3] [4]

Файл: mkdocs.yml

Сохрани в корне проекта:

```
site_name: Responsibility Response Model
site_description: Универсальная модель ответственности, задержки помощи и компенсации
site_author: Your Name
site_url: https://YOUR_USERNAME.github.io/responsibility-response-model/

repo_name: responsibility-response-model
repo_url: https://github.com/YOUR_USERNAME/responsibility-response-model

theme:
  name: material
  language: ru
  features:
    - navigation.tabs
    - navigation.sections
    - navigation.top
    - search.highlight
    - content.code.copy

nav:
  - Главная: index.md
  - Модель: model.md
  - Примеры: examples.md
  - Установка: installation.md

markdown_extensions:
  - admonition
  - pymdownx.highlight
  - pymdownx.superfences
  - pymdownx.arithmatex:
      generic: true
```

Файл: docs/index.md

```
# Responsibility Response Model

**Responsibility Response Model (RRM)** — универсальная модель ответственности.

Она оценивает не только решение в момент риска, но и:

- скорость реакции;
- компенсацию вреда;
- поддержку пострадавшего;
- исправление ошибки;
- ответственность во времени.

## Главная формула

\[
V(T)=B(T)-R(T)+\int_0^T e^{-\lambda t}S(t)dt
\]

Где:

- \(B(T)\) — ожидаемая польза;
- \(R(T)\) — ожидаемый вред;
- \(S(t)\) — поддержка и исправление;
- \(D(t)=e^{-\lambda t}\) — коэффициент задержки;
- \(T\) — период ответственности.

## Главный принцип

\[
\boxed{
\text{Чем позже помощь, тем меньше она искупает вред.}
}
\]

## Где применяется

- Человек, спасший другого.
- Государство во время кризиса.
- Финансовая система.
- Нейросеть после ошибки.
- Любая система, создающая риск.
```

Файл: docs/model.md

```
# Модель

## Формула

\[
V(T)=B(T)-R(T)+\int_0^T D(t)S(t)dt
\]
```

```
## Коэффициент задержки

\[
D(t)=e^{-\lambda t}
\]

## Поддержка

\[
S(t)=S_m(t)+S_f(t)+S_s(t)+S_p(t)+S_r(t)
\]

Где:

- \(S_m\) — материальная помощь;
- \(S_f\) — финансовая помощь;
- \(S_s\) — социальная поддержка;
- \(S_p\) — психологическая поддержка;
- \(S_r\) — исправление ошибки.

## Вывод

Мораль измеряется не в момент поступка, а во всей длине жизни после него.
```

Файл: docs/examples.md

```
# Примеры

## Человек-спасатель

- \(B(T)\) — спасённая жизнь.
- \(R(T)\) — инвалидность спасшего.
- \(S(t)\) — лечение, деньги, уход, признание.
- \(D(t)\) — скорость реакции общества.

## Финансовый кризис

- \(B(T)\) — стабильность системы.
- \(R(T)\) — дефолты и паника.
- \(S(t)\) — ликвидность, гарантии, компенсации.
- \(D(t)\) — скорость реакции регулятора.

## Нейросеть

- \(B(T)\) — польза ИИ.
- \(R(T)\) — ошибки и вред.
- \(S(t)\) — блокировка, исправление, компенсация.
- \(D(t)\) — скорость реакции разработчика.
```

Файл: docs/installation.md

```
# Установка

## Локальная установка

```bash
git clone [https://github.com/YOUR_USERNAME/responsibility-response-model.git](https
cd responsibility-response-model

python -m venv .venv
source .venv/bin/activate

pip install -e .
```

Запуск

```
rrm-run
```

Тесты

```
pytest
```

Документация локально

```
pip install mkdocs mkdocs-material
mkdocs serve
```

Сайт откроется по адресу:

```
http://127.0.0.1:8000/
```

```
## Обнови requirements.txt

```text
numpy
matplotlib
pillow
pytest
pytest-cov
mkdocs
mkdocs-material
mkdocstrings[python]
```

Как включить GitHub Pages

1. Открой репозиторий на GitHub.
2. Перейди в **Settings**.
3. Слева выбери **Pages**.
4. В разделе **Build and deployment** выбери:

```
Source: GitHub Actions
```

GitHub требует, чтобы источник GitHub Pages был установлен как `GitHub Actions`, чтобы custom workflow мог публиковать сайт.[3] [5]

Как задеплоить

```
git add .
git commit -m "Add documentation deployment workflow"
git push origin main
```

После push:

1. Открой вкладку **Actions**.
2. Увидишь workflow **Deploy documentation**.
3. Дождись зелёной галочки.
4. Открой URL, который GitHub покажет в job `deploy`.

Что произойдёт

- MkDocs соберёт Markdown в статический сайт.
- GitHub Actions загрузит папку `site` как artifact.
- `actions/deploy-pages` опубликует сайт.
- При каждом новом push в `main` документация обновится автоматически.

Финальная формула

$$\boxed{V(T) = B(T) - R(T) + \int_0^T e^{-\lambda t} S(t) dt}$$

Финальный принцип

> Документация — это часть ответственности проекта: она должна быть доступн

Теперь у проекта есть:

- код;
- тесты;

- CI;
- установка через `pip`;
- документация;
- автоматический деплой документации;
- публичный сайт на GitHub Pages. [1] [2] [3]

⁂

1. https://docs.github.com/en/pages/getting-started-with-github-pages/creating-a-github-pages-site
2. https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site
3. https://docs.github.com/en/get-started/start-your-journey/deploying-your-website-automatically
4. https://docs.github.com/api/article/body?pathname=/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages
5. https://docs.github.com/en/pages
6. https://github.com/marketplace/actions/mkdocs-action
7. https://github.com/marketplace/actions/mkdocs-build
8. https://github.com/marketplace/actions/gh-pages-deploy
9. https://github.com/peaceiris/actions-mkdocs-gh-pages
10. https://github.com/marketplace/actions/github-pages-action
11. https://github.com/peaceiris/actions-gh-pages
12. https://docs.github.com/en/pages/getting-started-with-github-pages/using-custom-workflows-with-github-pages
13. https://github.com/marketplace/actions/mkdocs-deploy
14. https://github.com/marketplace/actions/deploy-mkdocs
15. https://github.com/marketplace/actions/github-pages-deploy